

rest-sample-dubbo-provider Kullanıcı Rehberi

Bu rehber ilk kullanım içindir.

Amaç kısa ve nettir: REST consumer'ın çağıracağı plain Java Dubbo provider'ı çalıştırmak.

İçindekiler

1. Bu Proje Ne İşe Yarar?
2. Akış Nasıl Çalışır?
3. Ne Zaman Kullanılır?
4. Gerçek Hayat Senaryoları
5. Hızlı Başlangıç
6. Kopyala-Yapıştır Ayarlar
7. Provider Şekilleri
8. DB Ve Hikari Ayarları
9. Concurrency Kuralı
10. Sık Hatalar

Bu Proje Ne İşe Yarar?

rest-sample-dubbo-provider, Dubbo service implementasyonlarını barındırır.

Spring Boot kullanmaz. REST API açmaz.

İster hazır JSON döner, ister PostgreSQL/Hikari üzerinden veri okuyup command işler.

Akış Nasıl Çalışır?

Mimari akış

```
flowchart LR
  A["Dubbo Consumer"] --> B["Dubbo Provider"]
  B --> C["Service Implementation"]
  C --> D["HikariCP"]
  D --> E["PostgreSQL"]
```

Consumer sadece çağırır. DB bağlantısı provider içindedir.

Ne Zaman Kullanılır?

Senaryo	Bu proje uygun mu?	Neden
Consumer Dubbo üzerinden veri alacak	Evet	Ana kullanım budur.
Hazır JSON provider lazım	Evet	Catalog static profile uygundur.
PostgreSQL query provider lazım	Evet	DB query profile uygundur.
REST endpoint açılacak	Hayır	REST consumer tarafında açılır.
DB write command işlenecek	Evet	Customer command service örneği vardır.

Gerçek Hayat Senaryoları

Senaryo	Provider ne yapar?	Önerilen şekil
Katalog read	Hazır JSON üretir, DB kullanmaz.	<code>catalog-static-provider</code>
Müşteri listeleme	PostgreSQL'den müşteri listesi okur.	<code>db-query-provider</code>
Customer command	Create, patch, delete işlemlerini DB'ye yazar.	Default/full
K8s static servis	Consumer provider'a Service DNS ile gelir.	<code>registry-enabled=false</code>
ZooKeeper zorunlu	Provider kendini ZooKeeper'a register eder.	<code>registry-enabled=true</code>

Provider tarafında en önemli kural şudur: DB kapasitesini burada koruyun.

Consumer daha çok request gönderse bile provider Hikari ve method limitleriyle sistemi sınırlar.

Hızlı Başlangıç

PostgreSQL hazırsa provider'ı çalıştırın:

```
mvn -q package
java -jar target/rest-sample-dubbo-provider-0.3.2.jar
```

Default local ayar ZooKeeper istemez:

```
reactor.dubbo.registry-enabled=false
dubbo.provider.host=127.0.0.1
dubbo.provider.port=20880
```

Kopyala-Yapıştır Ayarlar

En küçük catalog provider:

```
reactor.dubbo.registry-enabled=false
dubbo.provider.service.default.max-concurrent=16
```

DB query provider:

```
sample.db.maximum-pool-size=2
sample.db.minimum-idle=0
dubbo.provider.service.CustomerQueryService.max-concurrent=2
dubbo.provider.service.CustomerQueryService.method.getDatabaseCustomersJson.max-concurrent=1
```

Command provider:

```
sample.db.maximum-pool-size=2
dubbo.provider.service.CustomerCommandService.max-concurrent=2
dubbo.provider.service.CustomerCommandService.method.createCustomer.max-concurrent=2
dubbo.provider.service.CustomerCommandService.method.patchCustomerSegment.max-concurrent=1
```

Provider executor için başlangıç ayarı:

```
dubbo.provider.executor.thread-pool=eager
dubbo.provider.executor.core-threads=1
dubbo.provider.executor.max-threads=8
dubbo.provider.executor.queue-capacity=16
dubbo.provider.executor.idle-timeout-ms=30000
dubbo.provider.executor.io-threads=1
```

Bu sınır, Dubbo handler thread sayısının yük altında yüzlerce thread'e çıkmasını önler. Create işlemleri bağımsız customer key'lerinde Hikari 2 kapasitesini kullanabilir. Patch/delete gibi aynı satırda yarışabilecek işlemler 1 olarak kalır.

ZooKeeper registration:

```
reactor.dubbo.registry-enabled=true
reactor.dubbo.registry-address=zookeeper://zookeeper-client.platform.svc.cluster.local:2181
reactor.dubbo.registry-root=dubbo
```

Provider Şekilleri

Şekil	Ne açar?	Ne zaman kullanılır?
catalog-static-provider	Sadece hazır catalog JSON	En küçük read-only provider
db-query-provider	Sadece DB query service	Command yoksa
Default/full	Catalog + customer query + command	Tüm sample endpoint'leri gerekiyorsa

DB Ve Hikari Ayarları

Property	Ne işe yarar?	Başlangıç
sample.db.jdbc-url	PostgreSQL bağlantı adresi	Ortama göre değişir.
sample.db.maximum-pool-size	Fiziksel DB connection üst limiti	2 iyi başlangıçtır.
sample.db.minimum-idle	Boşta tutulacak connection sayısı	Low-RSS için 0.
sample.db.connection-timeout-ms	Pool'dan connection bekleme süresi	Kısa tutun, DB yavaşsa ölçün.
sample.db.schema-init	Demo schema oluşturur	Production'da kapalı olmalıdır.

Docker PostgreSQL Checkpoint Başlangıç Reçetesi

Sample Docker Compose şu güvenli başlangıç değerlerini kullanır:

```
checkpoint_timeout=15min
checkpoint_completion_target=0.9
min_wal_size=256MB
max_wal_size=1GB
```

Bu ayarlar sürekli write yükünde checkpoint disk I/O baskısının REST p99 değerini büyütmesini azaltır. Commit güvenliği korunur. `fsync`, `synchronous_commit` ve `full_page_writes` açık kalır.

Harici PostgreSQL servisinde bu değerleri doğrudan kopyalamayın. Database ekibi disk gecikmesine, write hacmine ve recovery süresi hedefine göre ölçerek ayarlamalıdır. p99 değerini düşürmek için `synchronous_commit=off` kullanmayın.

Concurrency Kuralı

Provider limitleri Hikari kapasitesiyle uyumlu olmalıdır.

Interface	Property	Öneri
Query service	<code>dubbo.provider.service.CustomerQueryService.max-concurrent</code>	Hikari pool size veya daha düşük.
DB list method	<code>dubbo.provider.service.CustomerQueryService.method.getDatabaseCustomersJson.max-concurrent</code>	DB yavaşsa 1-2.
Command service	<code>dubbo.provider.service.CustomerCommandService.max-concurrent</code>	Hikari pool size veya daha düşük.

Interface	Property	Öneri
Create command	<code>dubbo.provider.service.CustomerCommandService.method.createCustomer.max-concurrent</code>	Bağımsız key'ler için Hikari 2 ile 2.
Patch/delete command	İlgili <code>method.<name>.max-concurrent</code> property	Aynı satırda yarışma varsa 1.

Client tarafında `c64` yük gelmesi Hikari'nin 64 connection açacağı anlamına gelmez.

Hikari 2 ise aynı anda iki DB işi çalışır. Geri kalan iş kısa süre bekler veya fail-fast olur.

Consumer baskısı	Provider Hikari	Doğru yorum
<code>c64</code>	2	64 request gelir, ama aynı anda 2 DB işi çalışır.
<code>c256</code>	4	256 request DB'ye aynı anda girmez. Queue ve bulkhead devrededir.
<code>balanced-wide</code>	2	Risklidir. Queue büyüyebilir, p99/RSS artabilir.
<code>micro-1x1</code>	1-2	Memory ve DB güvenliği için doğru başlangıçtır.

Sık Hatalar

Belirti	Muhtemel neden	Çözüm
Consumer <code>provider unavailable</code> alıyor	Provider kapalı veya adres yanlış.	Host, port ve static/ZooKeeper ayarlarını kontrol edin.
DB wait artıyor	Hikari pool dolu.	Önce SQL ve index kontrol edin, sonra pool artırın.
p99 saniyelere çıkıyor	Queue fazla büyüdü.	Provider method limitlerini DB kapasitesine indirin.
RSS yüksek	Full provider yüzeyi gereksiz olabilir.	<code>catalog-static-provider</code> veya <code>db-query-provider</code> kullanın.